

AI 使用说明情况

第二次挑战 · 姜佳岩 · 2026-10-03 更新

1 使用工具与提示词

使用 Codex 作为 AI 助手。本助手基于 GPT-6；当前记录未确认更具体的界面型号名称。交流主要使用中文，并附上终端截图。

源码学习时，我提出“你直接把该读的代码和解释整理，一定要适合我这种零基础的阅读和学习”，让 AI 按请求处理流程整理关键代码，并配上解释与源码链接。随后，我逐项询问“查找已有前缀 KV”“请求加入 waiting_queue”和“并发数是什么”，也贴出 req.init_next_round_input(self.tree_cache) 与 adder.add_one_req(...) 的代码，让 AI 解释缓存匹配和请求加入批次的过程。这些提问帮助我把术语、代码操作和实验中的现象联系起来。

2 AI 实际参与的工作

AI 帮助阅读第二次挑战要求，生成原生 /generate 的实验客户端脚本，设计两组配对 token 输入，加入版本检查、清空缓存、预热排除、SSE 解析和统计功能。客户端代码由 AI 编写；本人在 Ubuntu 中执行命令、运行实验并把原始结果复制到 Windows 工作目录。

AI 根据保存的逐请求文件和 SSE 事件重新核对输入、输出、命中 token、峰值并发、发送顺序及延迟分位数，生成 verification.json。报告表格来自真实 run-1 的汇总文件，没有将离线脚本检查的合成数据当作实验结果。

任务二的官方源码定位、请求主流程图、因果注意力分析及技术报告排版也使用了 AI 辅助。我参考讲义和源码链接阅读关键函数，再通过具体代码和术语提问，逐步理解请求从接收到返回结果的过程。

3 出现的问题与核验

在终端操作时，曾把结果展示行 "assignment_checks_passed": true 输入为命令，导致 command not found。结果行与执行命令的呈现没有被充分区分，AI 后续解释了这一点。该误操作发生在实验结束后，没有更改已保存的数据。

源码阅读时没有直接照抄题目中的函数名。AI 核对官方 v0.5.14 后发现实际前缀匹配入口是 Req.init_next_round_input → tree_cache.match_prefix，并说明 normal 与 overlap 调度分支的区别。这里记录的是核对结果，不声称对 SGLang 做过代码修改或 GPU 性能剖析。

4 源码学习与自查

我利用讲义里的源码链接找到对应代码，一边看解释，一边阅读关键函数。看完后，再用四个小问题检查自己的理解。遇到不能说明白的地方，就回去看对应代码和讲义，继续向 AI 提问。这样能发现哪些内容只是阅读时觉得理解了，哪些内容已经能够自己解释。

数据与复现依据：src/target1/experiment.py、src/target1/workloads/、results/target1/ 下的原始事件、逐请求结果和汇总；源码依据为官方 v0.5.14，具体链接保存于 src/target2/source-map.md。